

# Machine learning without (or lack of) labels

Linus Aronsson

Materials have been developed from many sources  
e.g., Richard Johansson, Barbara Plank, Selpi

# Learning paradigms in ML



Expensive to get!

- **Supervised learning**
  - Goal: to learn mapping from examples to labels
- **Unsupervised learning**
  - No labels available
  - Goal: to summarize, explore, visualize, understand, discover structure of data
- **Semi-supervised learning**
  - Very small labelled data, large unlabelled data
- **Self-supervised learning**
  - Construct labels automatically from input (no manual labelling required)
- **Active learning**
  - Iteratively label most informative data points in a cost-sensitive manner

# Unsupervised learning (no labels)

- **Clustering**
  - Identify groups of similar data points
- **Dimensionality reduction**
  - Learn a new representation of the data (e.g., in lower dimensional form)
  - PCA, TSNE
- **Model statistical distribution of data**
  - Generative modelling
  - Anomaly detection

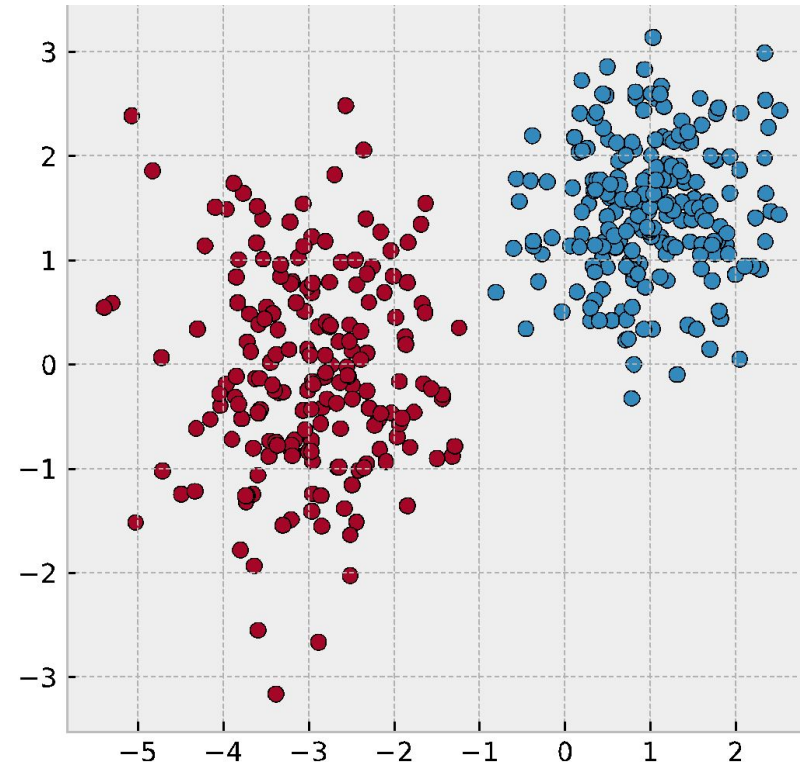
# Examples of successful applications of unsupervised learning

- **Cyber security field (anomaly detection)**
  - An example: Darktrace, a cyber security AI company
  - Read [the news](#) that mentions Darktrace uses an ensemble of 60 unsupervised models in 2018
- **Recommender systems**
  - Used by e.g. e-commerce and video streaming businesses
- **Social network analysis**
  - Analyzing polarization, trust, and conflict in social systems
  - Crucial for understanding online discourse, political divisions, and trust dynamics
  - Used to detect spread of misinformation

# Introduction to Clustering

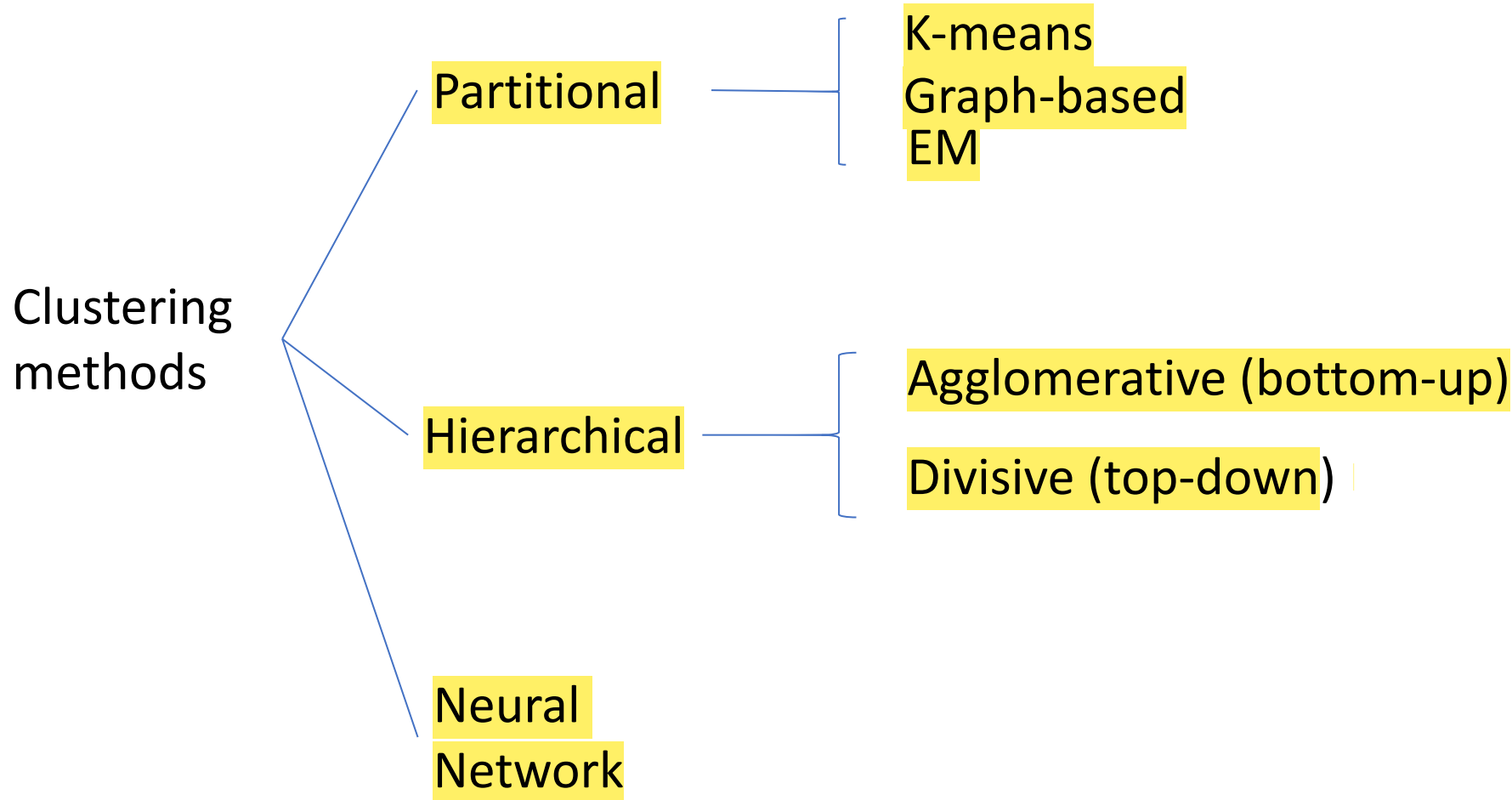
# Clustering

- Goal: to discover structure in the data or natural groups which are cohesive internally and different from the other group(s).
- Useful for summarizing, exploring, visualizing, and understanding data



How?

# Different clustering methods



# K-means

Input: K (#clusters) & training data

Algorithm:

- Randomly initialize K cluster centroids
- Repeat until converged:
  - Assign cluster to each training data, based on distance
  - Move each centroid to the center of its cluster

$$\arg \min_S \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2$$



# Clustering evaluation

Why is clustering hard?

Difficult to interpret and evaluate the results

1. **External evaluation:** Compare with ground-truth clustering. Examples: Rand index, Purity score
2. **Internal evaluation:** Measure quality of clustering based on structure in feature space. Example: silhouette score

See

<https://scikit-learn.org/stable/modules/clustering.html#clustering-performance-evaluation> for different performance metrics for clustering.

# Silhouette score

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

where

- ▶  $a_i$  is the average distance to other members in the **same** cluster
- ▶  $b_i$  the minimal average distance to **another** cluster

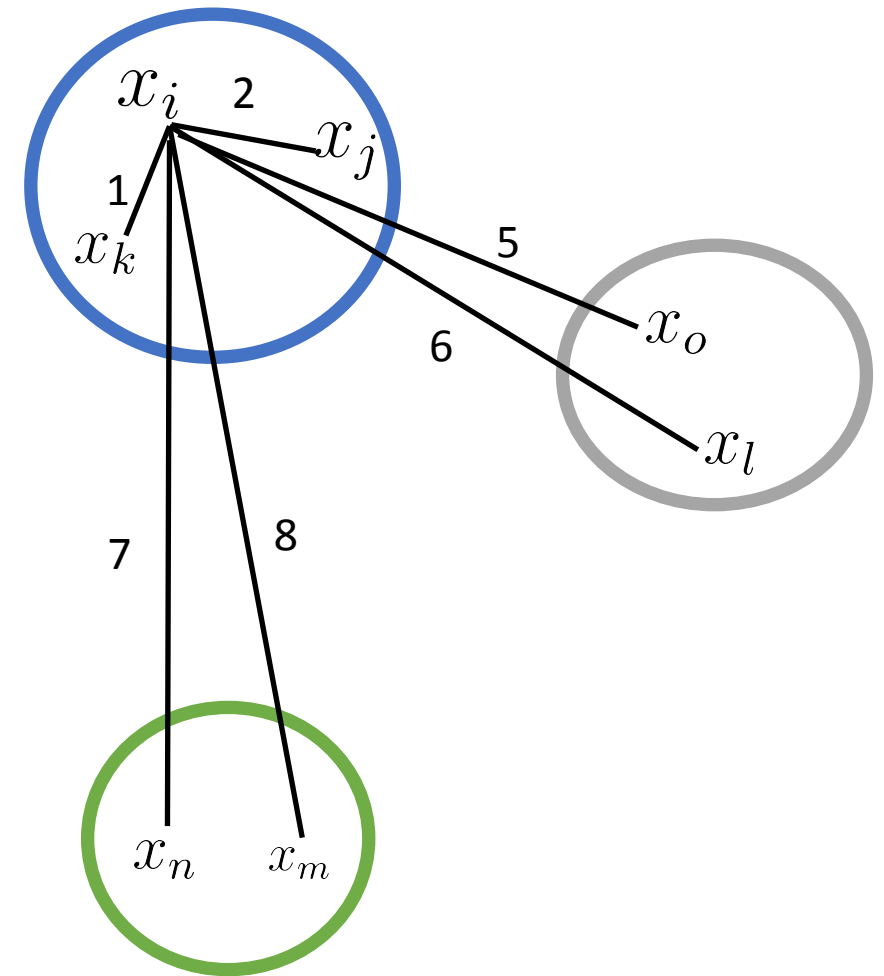
To calculate silhouette coefficient for only one sample:

$$a_i = (1 + 2)/2 = 1.5$$

$$b_i = \min((5 + 6)/2, (7 + 8)/2) = 5.5$$

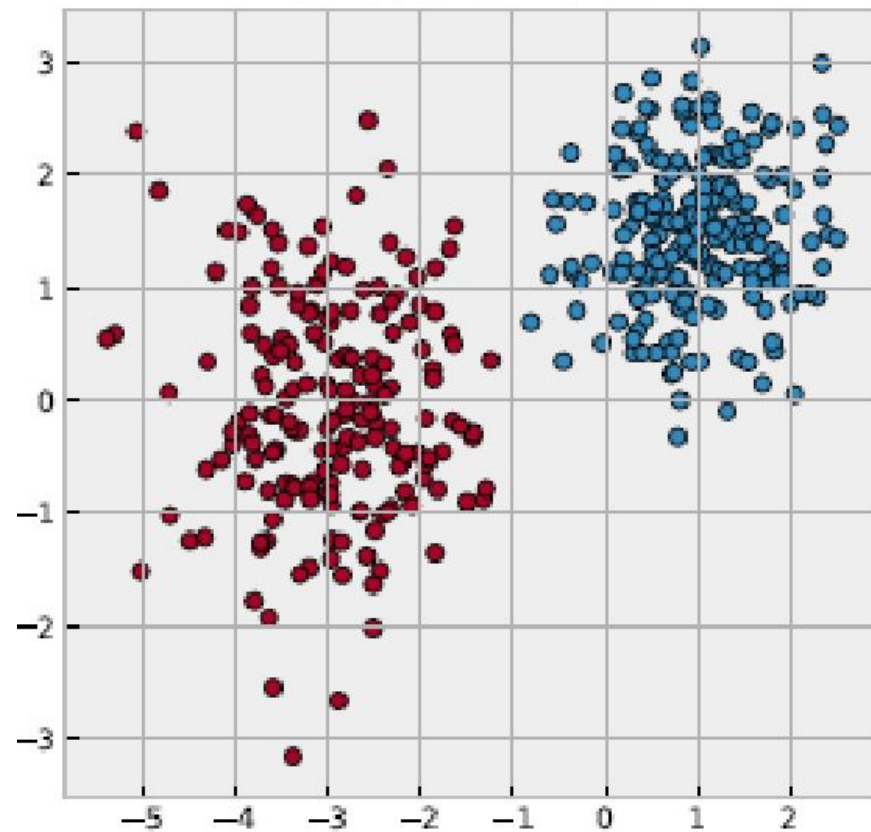
$$s_i = (5.5 - 1.5)/5.5 = 4/5.5$$

Range: [-1, +1]

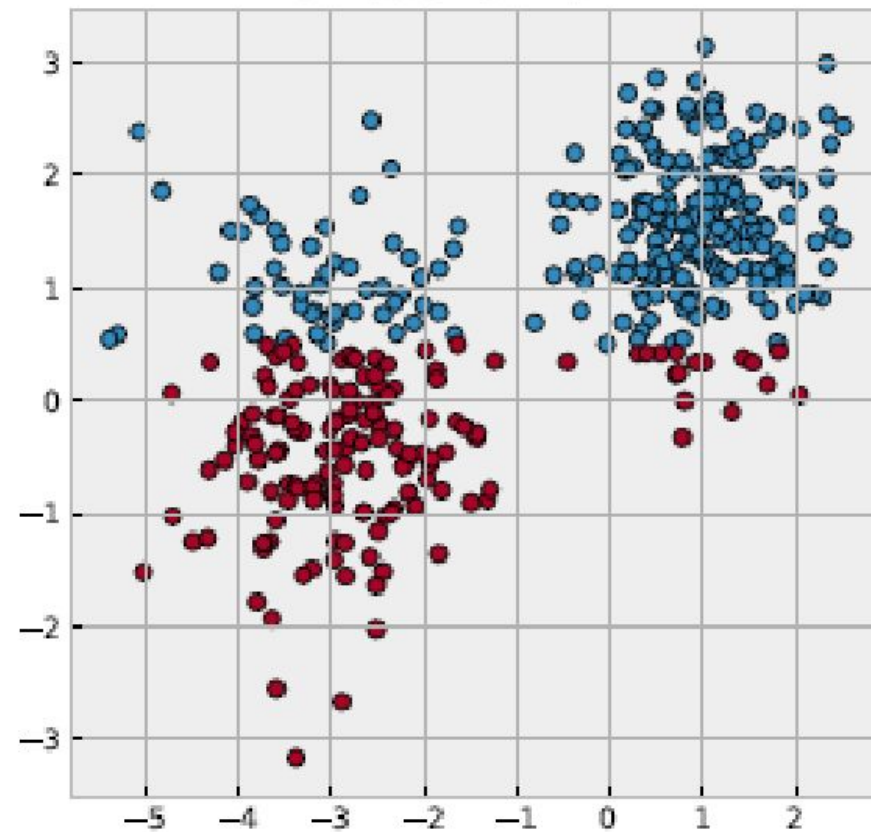


In sklearn, the `silhouette_score` calculates the mean Silhouette coefficient for all samples (not only one sample like the example here).

silhouette = 0.68



silhouette = 0.42



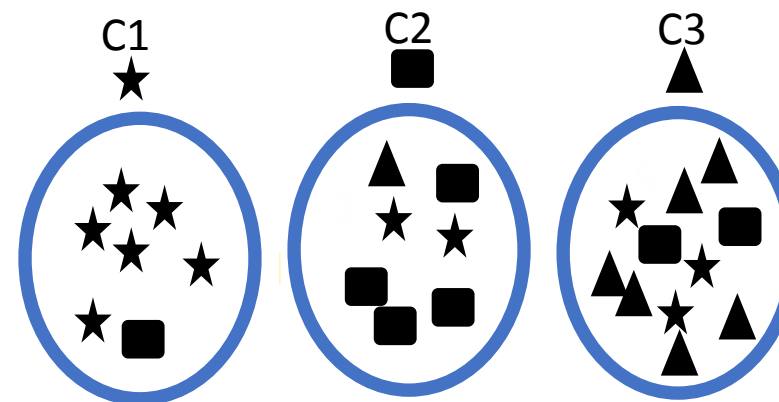
# Purity score

$$\text{Purity} = \frac{1}{N} \sum_{k=1}^K \max_{i=1}^{|C|} |S_k \cap C_i|$$

Example to calculate purity score:

- Assume we have clusters S1, S2, and S3.
- For each of the categories C1, C2, C3, assign member type with max count as ground truth. This gives C1 stars, C2 rectangles, C3 triangles.
- Create confusion matrix between the clusters S1, S2, S3, and the categories C1, C2, C3.
- Sum the correct numbers that are correctly classified and divide by the total of examples. Purity = (6+4+6)/25 = 16/25

For each C1, C2, C3, assign member type with max count. This gives C1 stars, C2 rectangles, C3 triangles



Blue circles are results of clustering

|    | C1 | C2 | C3 |
|----|----|----|----|
| S1 | 6  | 1  | 0  |
| S2 | 2  | 4  | 1  |
| S3 | 3  | 2  | 6  |

# K-means (2)

How to determine K?

Elbow method, trial and error, domain knowledge

## Advantages and disadvantages

- + Simple, easy to understand
- + Fast, robust enough
- + Good for distinct clusters
- Rely on K
- Can have local minima, depends on initialization
- Sensitive to outliers
- Sensitive to scaling

See demo

# K-means

In general, the results of the different runs could be different, as the alg. starts with random initialization

- If we want the alg. to always give the same results, set the random seed.
- No guarantee to find global optimum, depends on initialization

If elbow method does not give a clear indication, do for example:

- Plot the clusters and see if the clustering makes sense or
- use silhouette score

To improve the clustering results:

- Run the alg. with different centroid initialization
- Adjust number of iterations
- Find out the optimal number of clusters - do elbow method, silhouette score, check the cluster visualization

# K-means

## Termination conditions

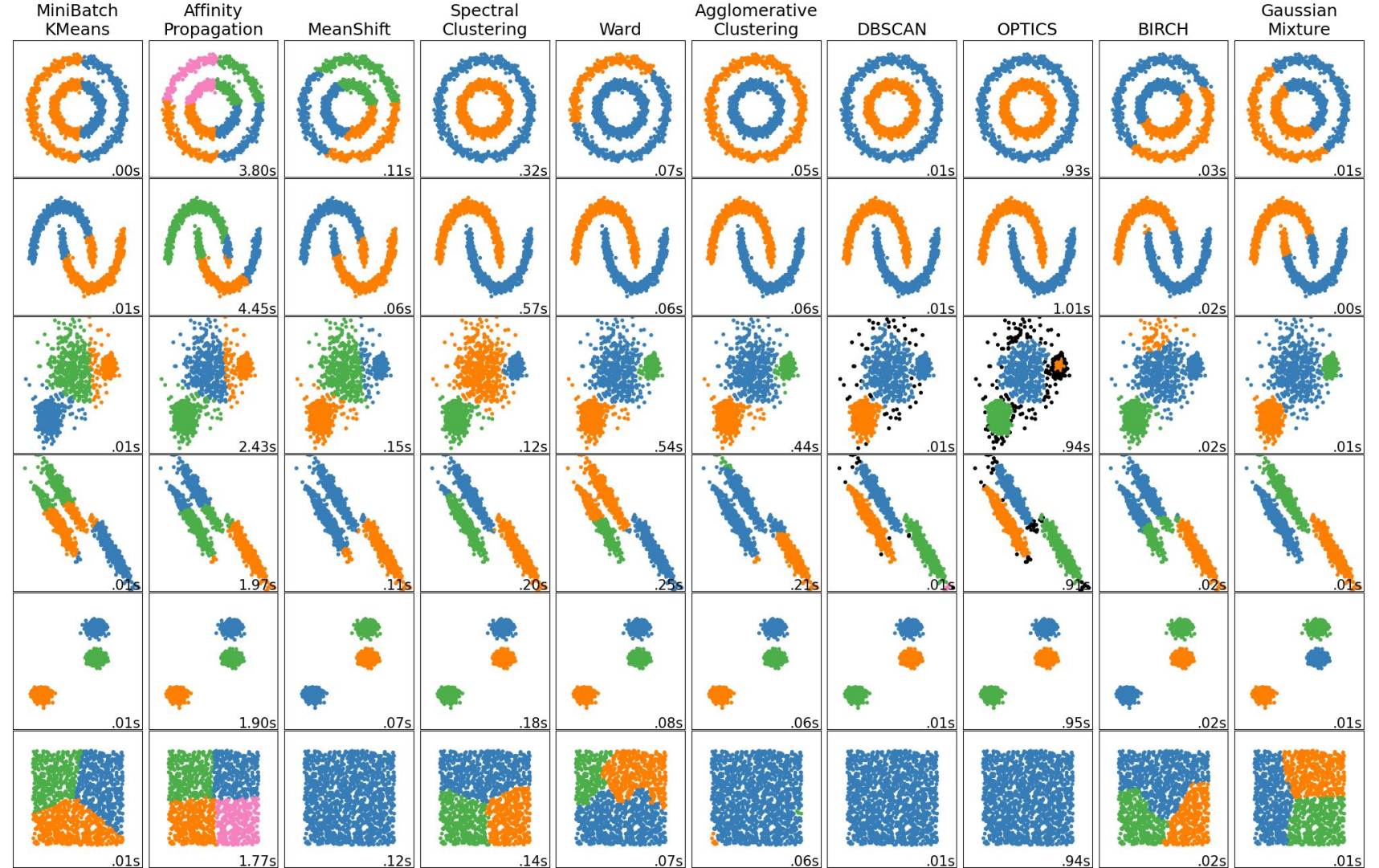
- Has reached the number of iterations set
- If the error is lower than a threshold used
- If the centroids do not change after the next iterations
- If assignment of samples to clusters does not change



# K-means

Doesn't work well for e.g.

- Data with different densities
- Outliers
- Non-convex shape





# DBSCAN

## Parameters:

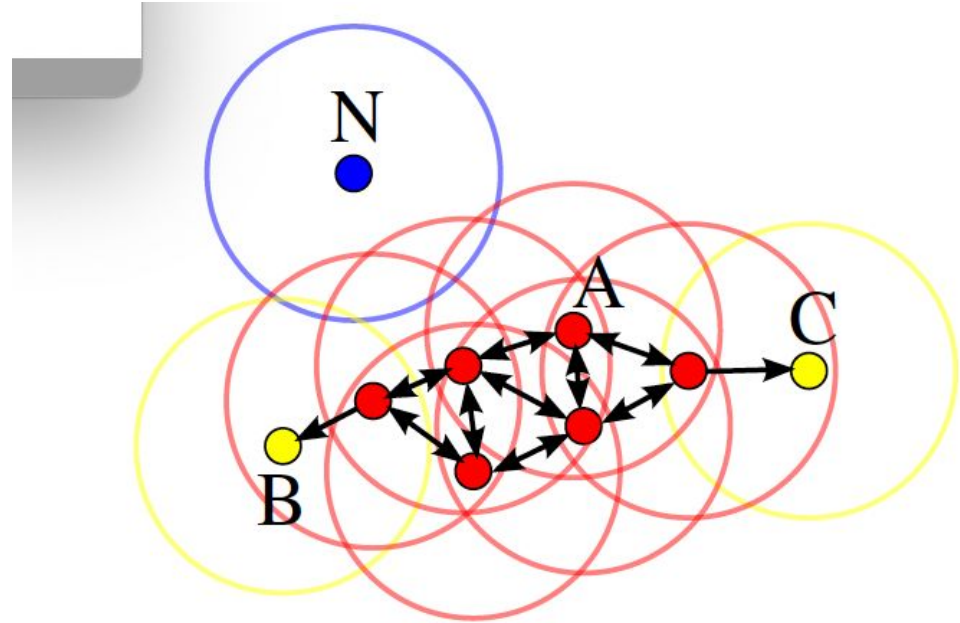
- eps
- Min\_samples

## Characteristics:

- Very sensitive to the parameters
- Can use any distance metric
- Can detect outliers

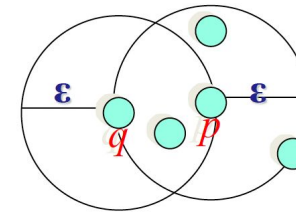
## In the figure:

- A (core point),
- B (border point),
- C (border point),
- N (noise point)



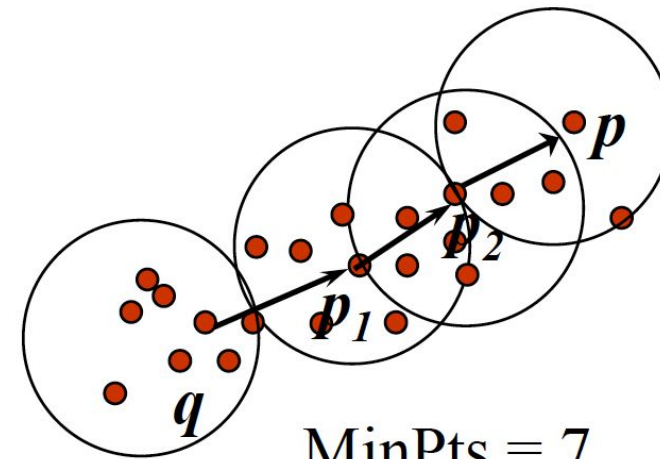
# DBSCAN

A point  $q$  is **density reachable** from another point  $p$ , if  $q$  is in the  $\epsilon$ -neighborhood of  $p$ , and  $p$  is a **core point**. This means that  $p$  must have at least  $\text{min\_samples}$  points within  $\epsilon$  distance (including  $p$  itself).



MinPts = 4

$p$  is **not** density-reachable from  $q$   
 $q$  is density-reachable from  $p$



MinPts = 7

$p$  is density-reachable from  $q$   
 $q$  is **not** density-reachable from  $p$

# DBSCAN

Parameters:  $\epsilon$  and  $\text{min\_samples}$

Algorithm:

- Randomly select a point  $P$ .
- Retrieve all points directly density-reachable from  $P$  w.r.t  $\epsilon$  (radius to search for neighbors).
- If  $P$  is a **core point**, a cluster is formed. Find recursively all its density connected points and assign them to the same cluster as  $P$ .
- If  $P$  is not a core point, assign  $P$  as an **outlier**, and iterate through the remaining unvisited points in the dataset.

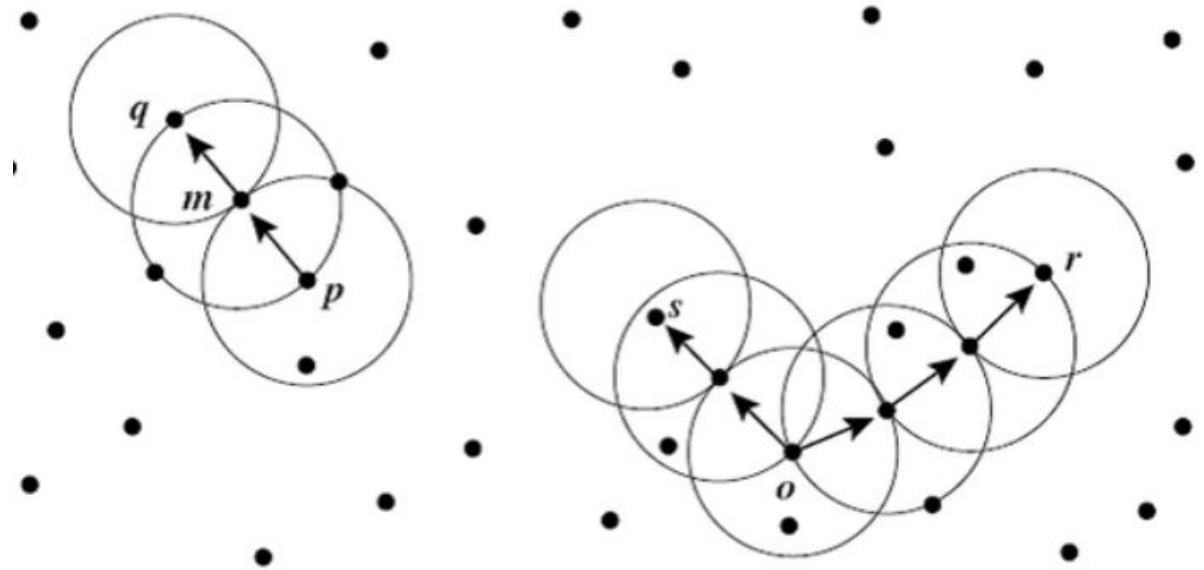


Figure from: [Alvin Thai at quora.com](#)

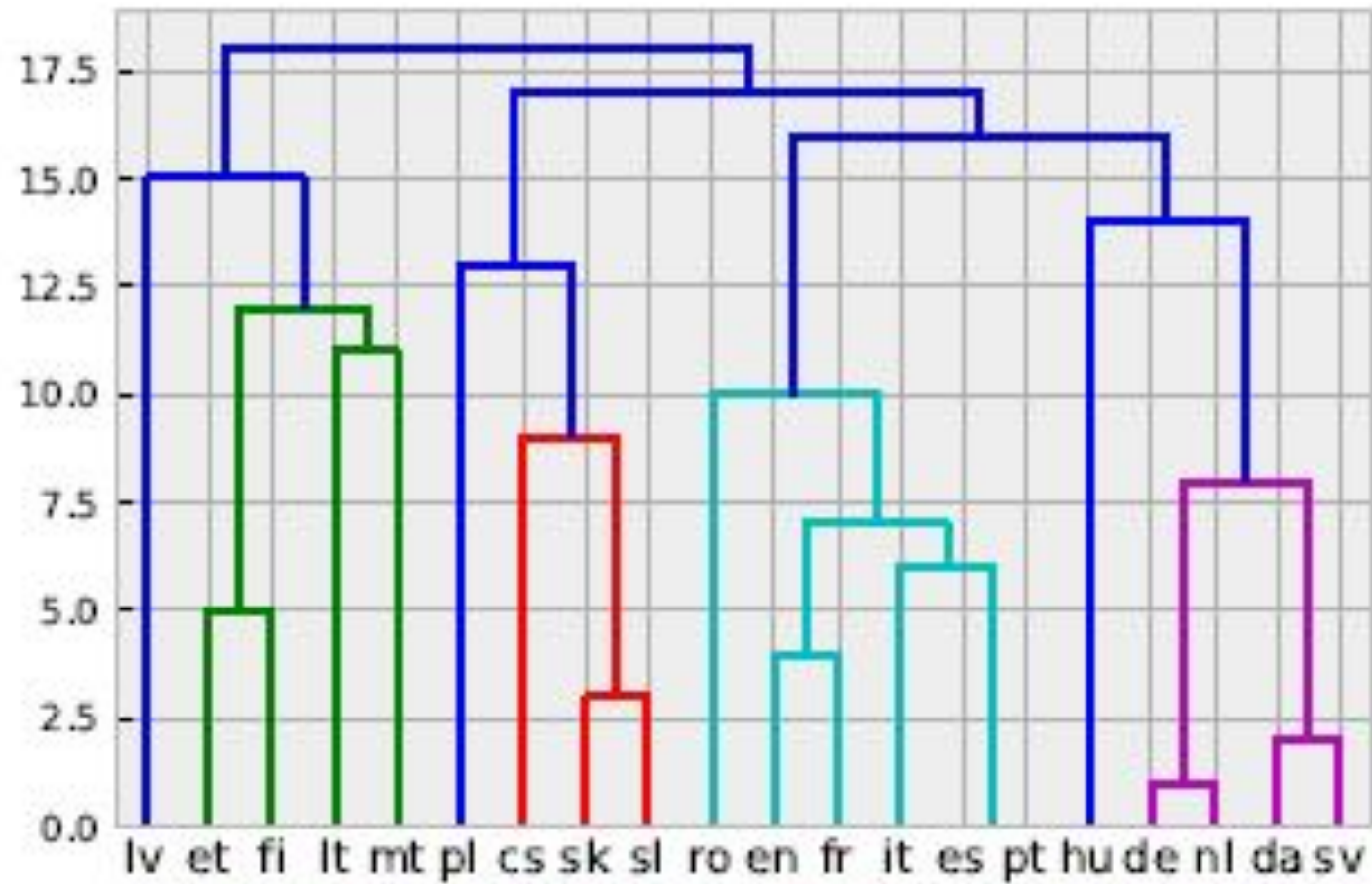
# DBSCAN

See demo

# Hierarchical clustering

- **Agglomerative**: This is a *bottom-up* approach in which each data point initially starts in its own cluster. Clusters are then iteratively merged based on their similarity, gradually forming larger clusters as one moves up the hierarchy.
- **Divisive**: This is a *top-down* approach where all data points begin in a single cluster. The algorithm then recursively splits clusters into smaller ones based on dissimilarity, progressively refining the structure as it moves down the hierarchy.

# Hierarchical clustering



# Agglomerative clustering

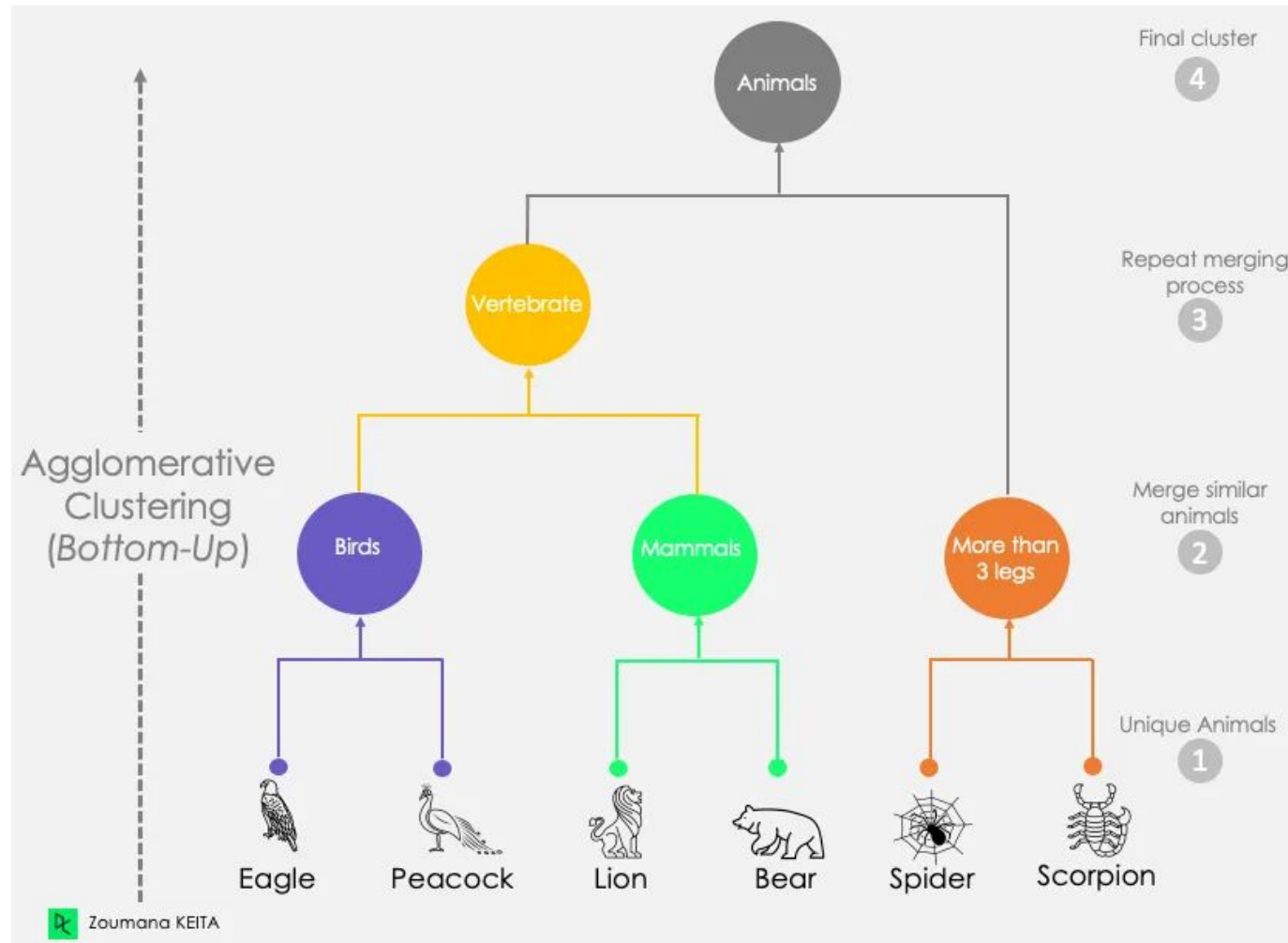


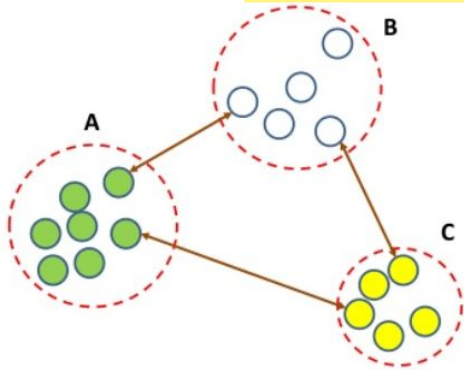
Image source:

<https://medium.com/@prasanth32888/agglomerative-hierarchical-clustering-ahc-e9e7a48cb042>

# Linkage criterion

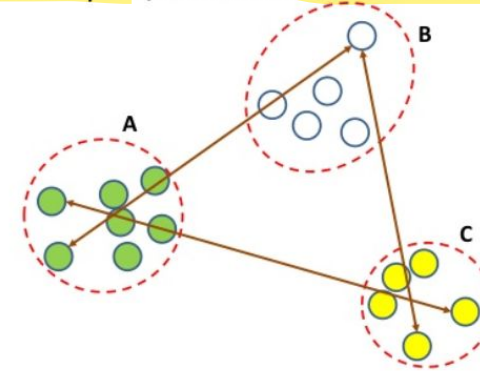
## Single-linkage clustering (SLINK)

- distance between two clusters is determined by a *element pair*, that are closest to each other



## Complete-linkage clustering (CLINK)

- distance between two clusters is determined by a *element pair*, that are farthest to each other





# Anomaly detection (outlier detection)

- Goal: detect if new data point is an outlier
- Statistical based
- ML based (decision tree, rule-based)
- Neural network based (autoencoders)

Applications: fraud detection, cyber security, manufacturing processes, monitoring machine

# Anomaly detection

Anomaly detection: Very small number of positive examples, large number of negative examples, future anomalies are unlikely to be similar to the training set

# Anomaly detection

How to evaluate anomaly detection algorithm:

- True positive, false positive, false negative, true negative
- Precision, recall
- $F_1$ -score

What about using accuracy?

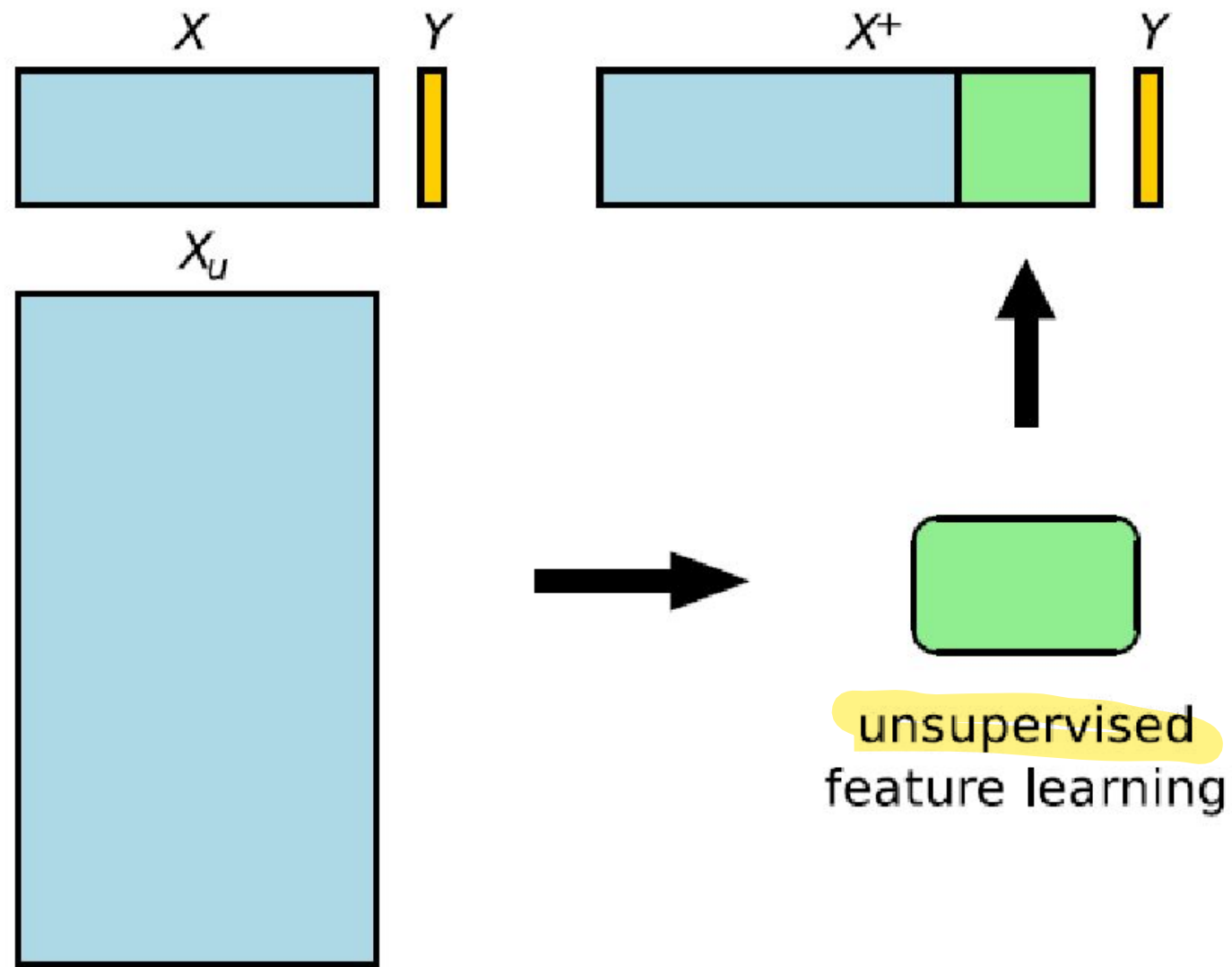
# Semi-supervised learning

Learns from both labelled and unlabelled data.

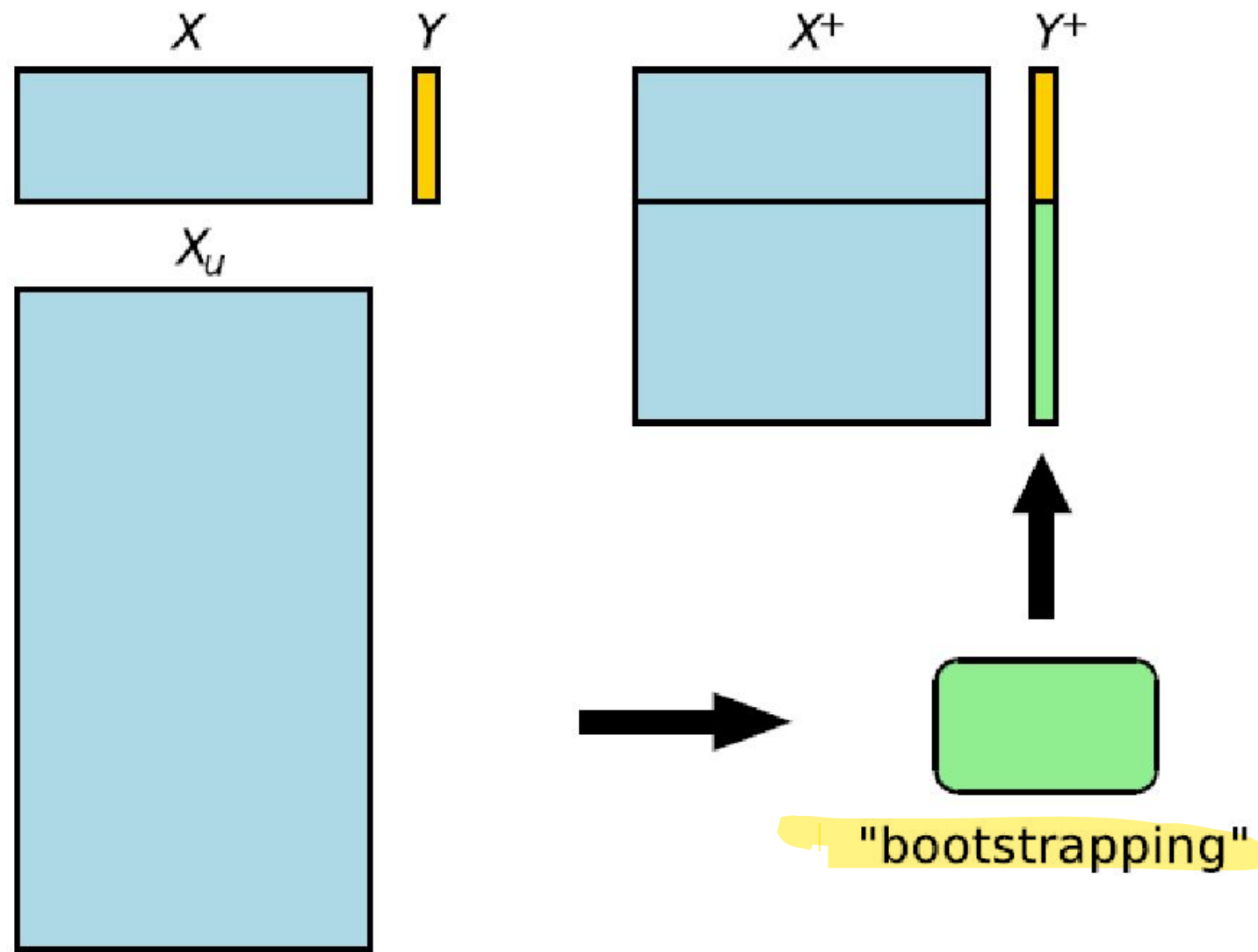
Main approaches:

- To add new features
- Train the system to label new instances automatically

# adding new features



# generalizing from instances



# Bootstrapping methods

A widely used SSL bootstrapping algorithm:  
**self-training**



# Self-training

- Input: labeled data  $\{(\mathbf{x}_i, y_i)\}_{i=1}^l$ , unlabeled data  $\{\mathbf{x}_j\}_{j=l+1}^{l+u}$ .

labeled  
data

unlabeled  
data

- Procedure:

1. Initially, let  $L = \{(\mathbf{x}_i, y_i)\}_{i=1}^l$  and  $U = \{\mathbf{x}_j\}_{j=l+1}^{l+u}$ .
2. Repeat:
3.     Train  $f$  from  $L$  using supervised learning.
4.     Apply  $f$  to the unlabeled instances in  $U$ .
5.     Remove a subset  $S$  from  $U$ ; add  $\{(\mathbf{x}, f(\mathbf{x})) | \mathbf{x} \in S\}$  to  $L$ .



# Self-training

- Procedure:
  1. Initially, let  $L = \{(\mathbf{x}_i, y_i)\}_{i=1}^l$  and  $U = \{\mathbf{x}_j\}_{j=l+1}^{l+u}$ .
  2. Repeat:
  3.       Train  $f$  from  $L$  using supervised learning.
  4.       Apply  $f$  to the unlabeled instances in  $U$ .
  5.       Remove a subset  $S$  from  $U$ ; add  $\{(\mathbf{x}, f(\mathbf{x})) | \mathbf{x} \in S\}$  to  $L$ .
- Parameters, e.g., iterations, pool/growth size, select
- Questions:
  - Q1: This is called a wrapper method. Why?
  - Q2: Why might this help to build a better system?
  - Q3: What might go wrong?

# Self-training: Summary

Q1: Wrapper?

choice of  $f$  left open

Q2: Works  
when?

broad margin, expected low error

Q3: Limitations?

errors get reinforced

Variants?

Yes, many, e.g., delible self-training, weigh instances,...

# Another way of doing self-training

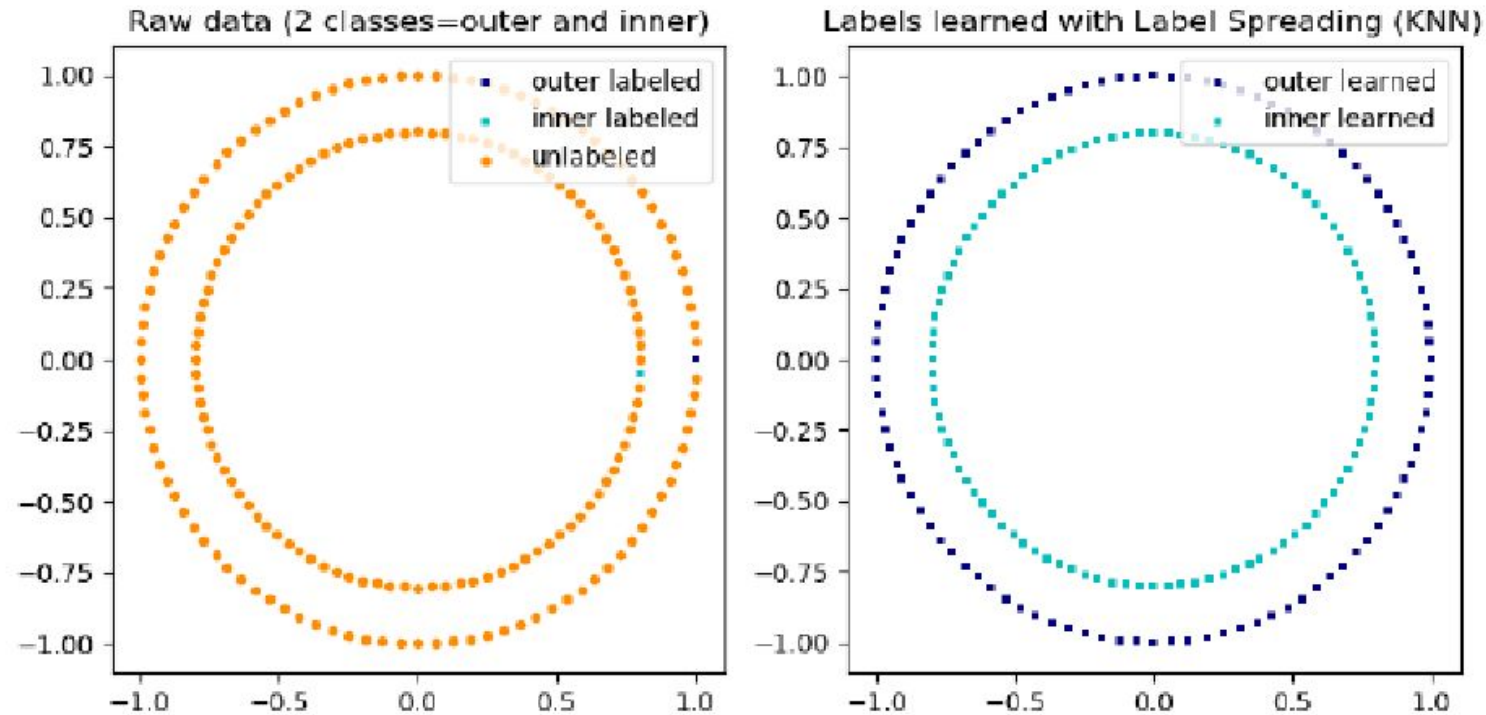
Use a weighted multi-objective function

- Train on both labelled and unlabelled data
- Use lower weight for the unlabelled data that are labelled by the trained model

# Co-training

- Two different views of the data are used to build the model.
- Two different feature sets that provide different, complementary information about the instance are needed.

# label propagation



[http://scikit-learn.org/stable/auto\\_examples/semi\\_supervised/plot\\_label\\_propagation\\_structure.html](http://scikit-learn.org/stable/auto_examples/semi_supervised/plot_label_propagation_structure.html)

# Self-supervised learning

- **Learns from unlabeled data** by **creating artificial labels** (pretext tasks) from the data itself.
- **Bridges unsupervised and supervised learning** by using supervision signals derived from the input data.
- **Pretext tasks** encourage the model to **learn meaningful representations** (e.g., predicting a missing patch, rotation angle, or future frame).
- **Downstream tasks** (e.g., classification, detection) benefit from these learned representations with **minimal labeled data**.
- **Widely used in NLP and computer vision**, such as LLMs (predict next word) or SimCLR (contrastive learning with augmentations).

# Self-supervised learning

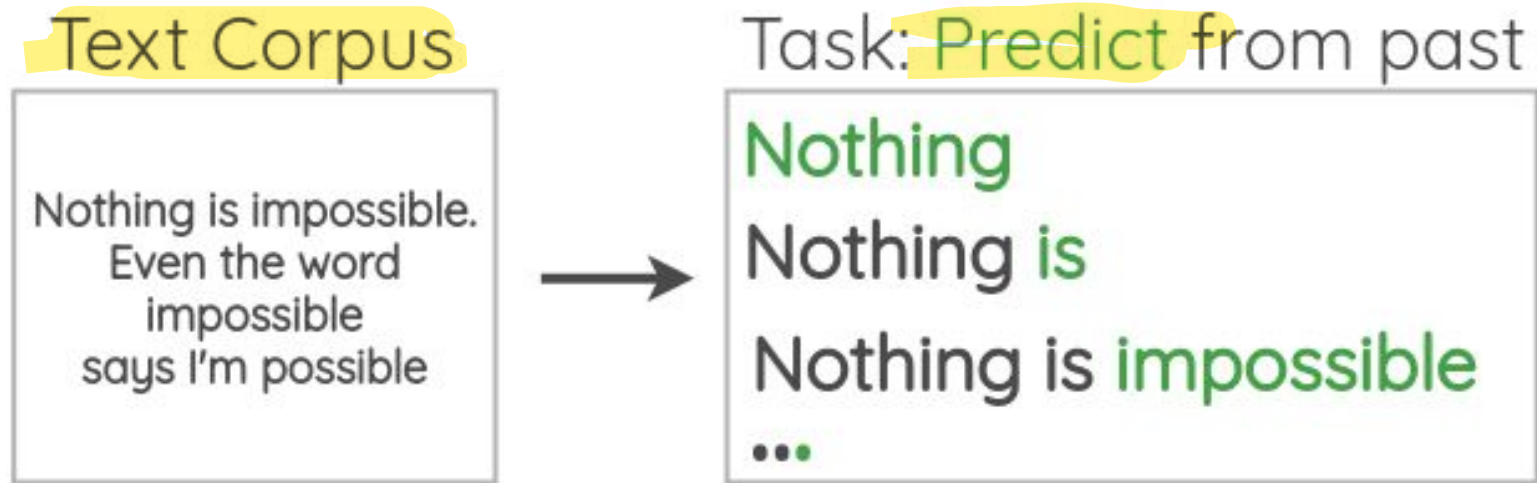


Image source: <https://amitnesh.com/posts/self-supervised-learning-nlp>

# Active learning

- Supervised learning is traditionally performed **passively**.
  - Train on all available data (features and labels)
- Labels are expensive to obtain
- Part of the input information may be noisy and misleading
  - Should ideally be excluded from training
- Active learning enables the learning algorithm to actively select the most informative data points to (1) include in the training set and (2) request labels for.



# Active learning

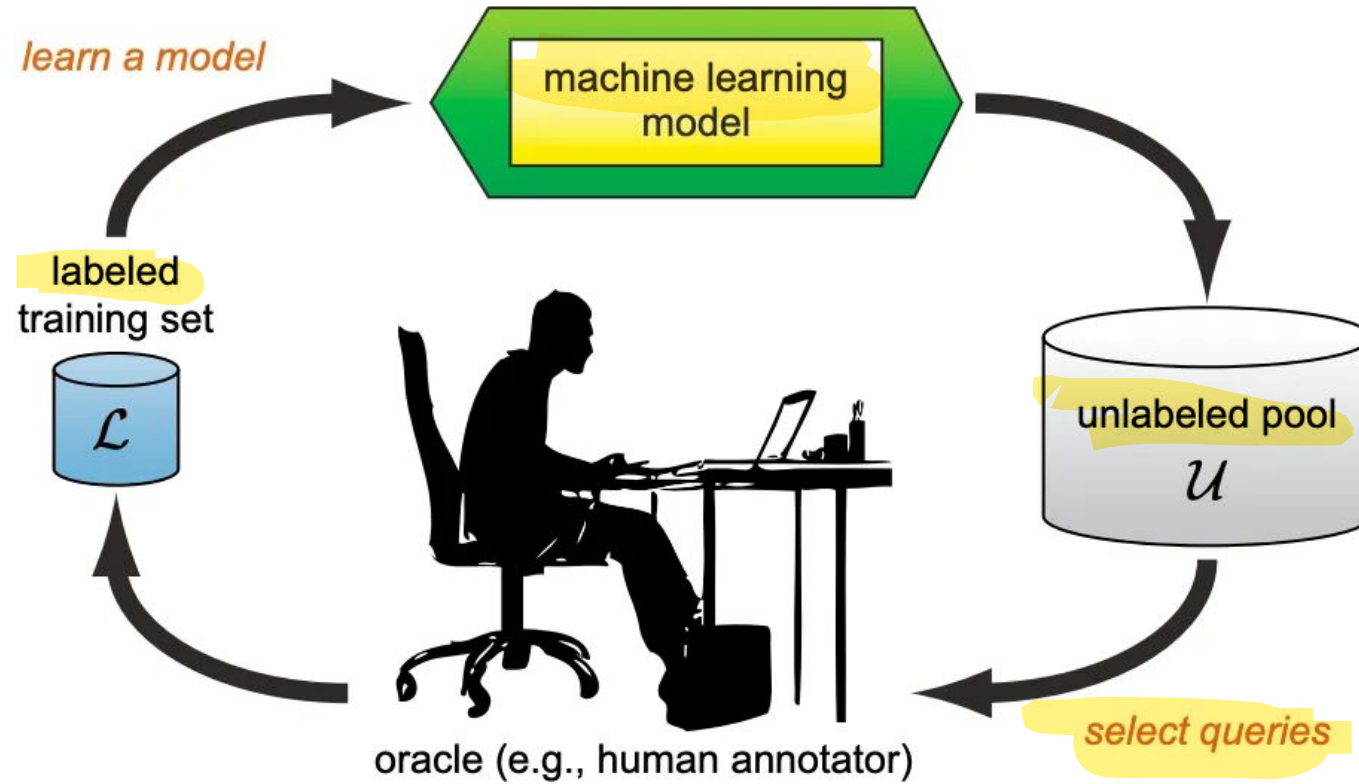


Image source: Burr Settles. Active learning literature survey, 2009.

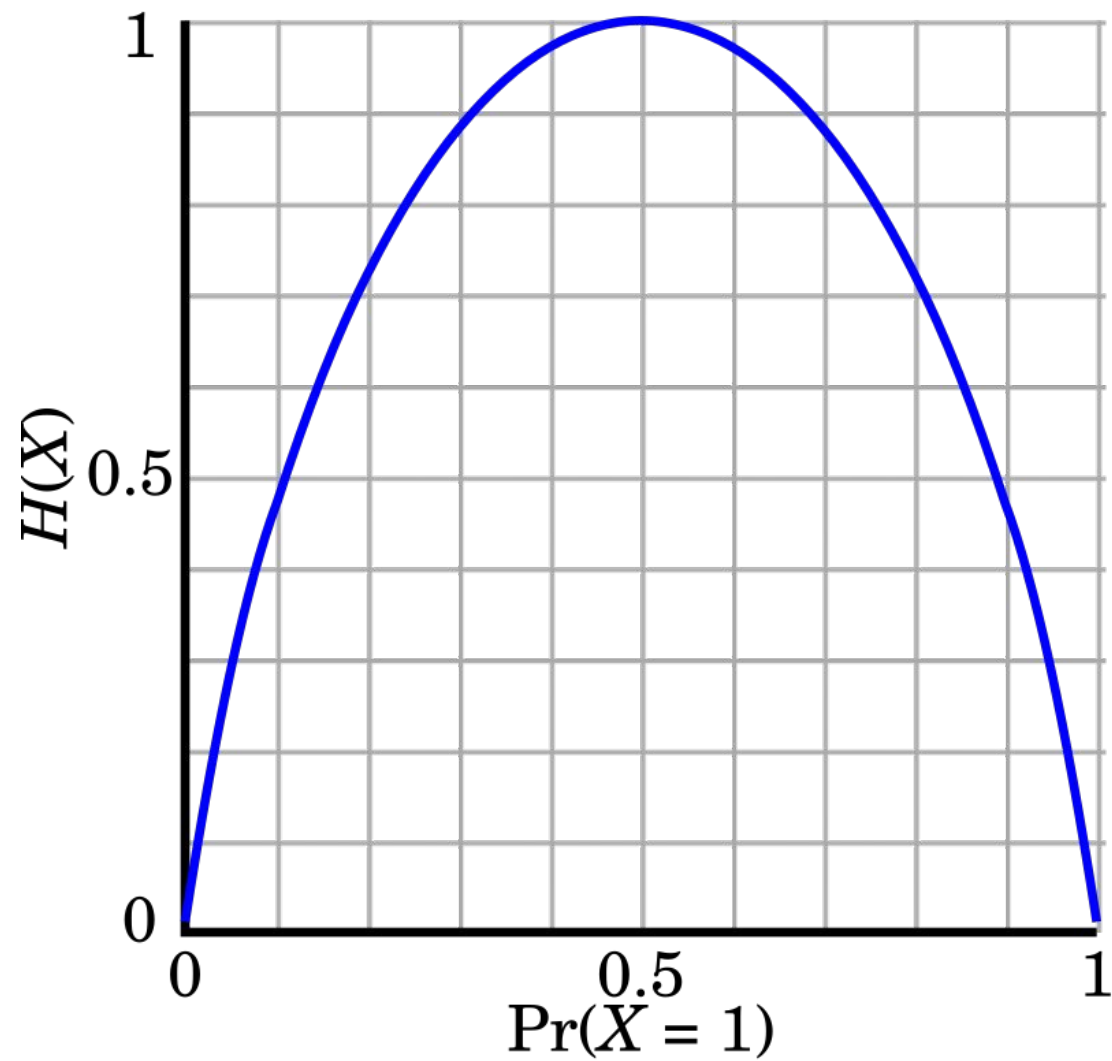
# Query strategies

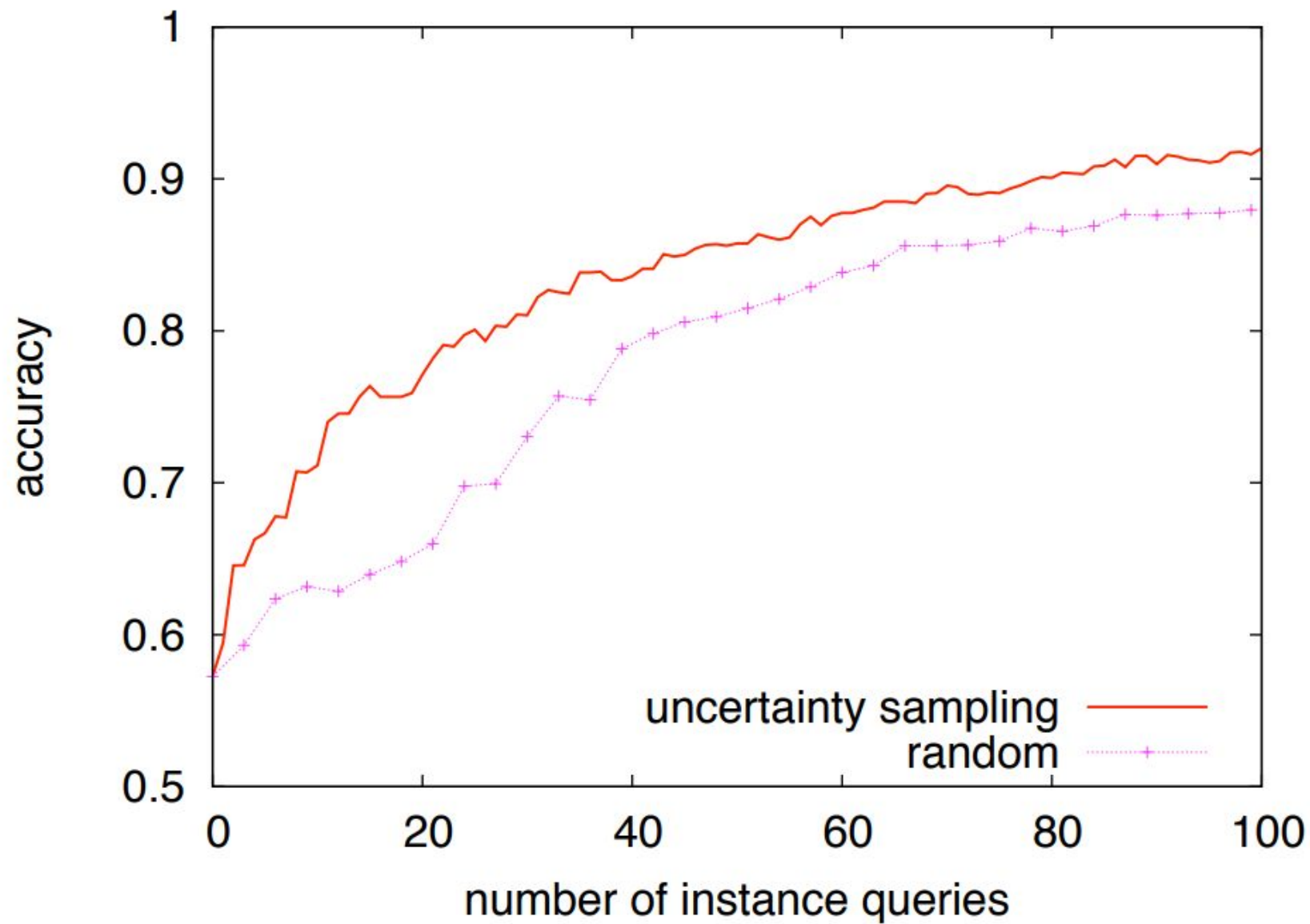
- **Uncertainty sampling**
  - Query the most **uncertain** unlabeled data points according to the model
- **Query by Committee**
  - Train multiple models on labeled data, query unlabeled data points where the models **disagree** the most
- **Geometric approaches**
  - Query unlabeled data points which are different from labeled data points (**uncertain**), while being **representative** of the data distribution (i.e. avoid selecting outliers)
  - Based on e.g. distance in feature space
- Other methods exist (including methods that combine the ideas above)
  - See survey in <https://burrsettles.com/pub/settles.activelearning.pdf>

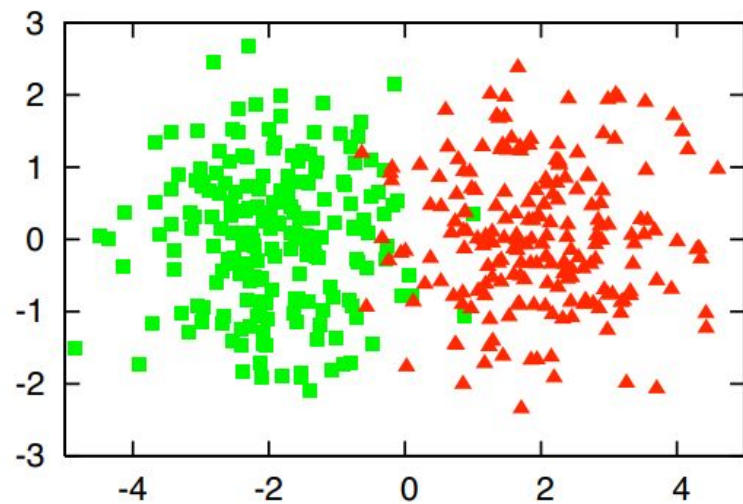
# Uncertainty sampling

- Straightforward for probabilistic classification models that output a probability distribution of classes  $p(y|x)$  (e.g., softmax output of neural network).
- Quantify uncertainty using entropy  $H(y) = - \sum_{i=1}^k p(y_i | x) \log p(y_i | x)$
- Assume 4 classes (e.g. dogs, cats, cars and planes for image classification). Model  $p(y|x)$  outputs softmax distribution over the 4 classes:
  - $[0.25, 0.25, 0.25, 0.25] \rightarrow \text{Entropy} = 2$
  - $[0.7, 0.1, 0.1, 0.1] \rightarrow \text{Entropy} = 1.35$
  - $[1, 0, 0, 0] \rightarrow \text{Entropy} = 0$
- Query unlabeled data point with largest entropy

## Entropy for 2 classes

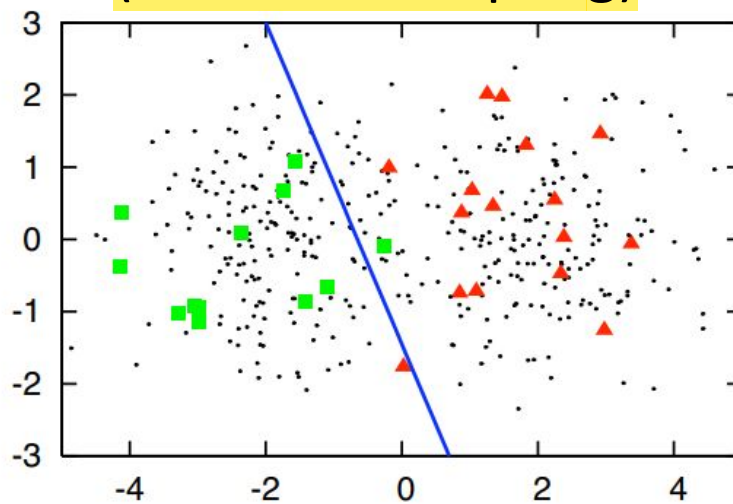






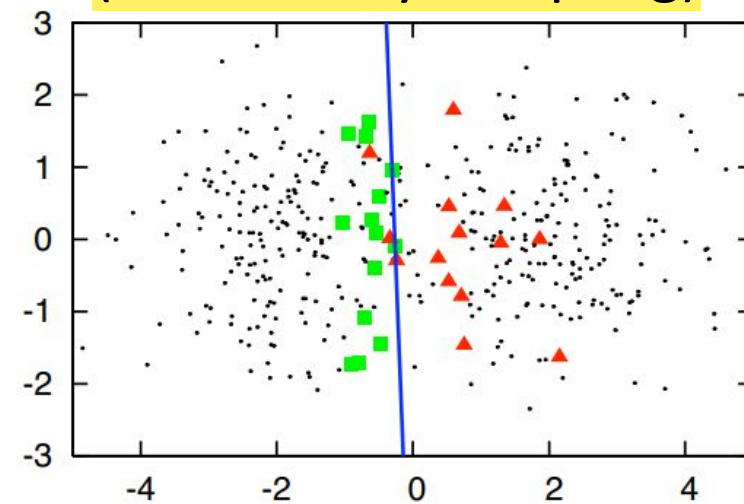
(a)

Passive learning  
(Random sampling)



(b)

Active learning  
(uncertainty sampling)

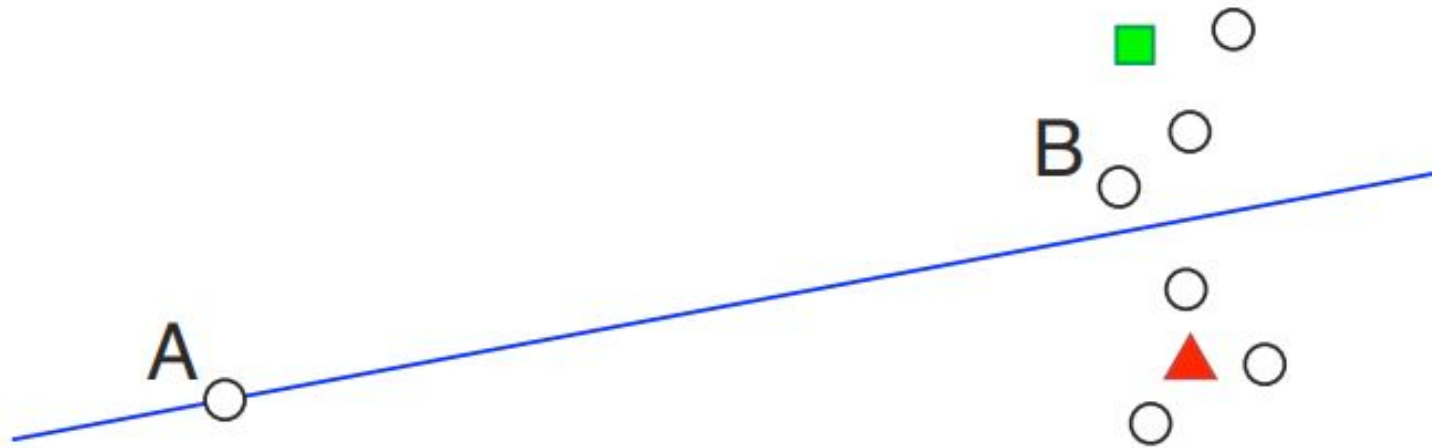


(c)

Image source: Burr Settles. Active learning literature survey, 2009.

# Selection bias

- Uncertainty sampling may select A over B, since both are close to the decision boundary
- But B is clearly more informative, since A is an outlier
- Fix: encourage representativeness in the query strategy
- An ideal query strategy combines uncertainty and representativeness



Burr Settles. Active learning literature survey, 2009.